

```
import numpy as np
import pandas as pd
import os
from sklearn.ensemble import AdaBoostRegressor
from sklearn.tree import DecisionTreeRegressor

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, StackingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import ConfusionMatrixDisplay
# Visualisation libraries
import matplotlib.pyplot as plt
# %matplotlib inline
import seaborn as sns

import plotly.express as px
import plotly.graph_objects as go
```

```
dfi=pd.read_csv('air_quality_with_aqi.csv')

# # Extract delhi's data

# df=city_day_data.groupby('StationId').get_group('DL001')

dfi.head()
```



	Timestamp	PM2.5 (µg/m³)	PM10 (µg/m³)	NO (µg/m³)	NO2 (µg/m³)	NOx (ppb)	NH3 (µg/m³)	SO2 (µg/m³)	CO (mg/m³)	Ozone (µg/m³)	...	SR (W/mt2)	BP (mmHg)	VWS (m/s)	AQI_PM2.5	AQI_
0	2024-01-01 00:00:00	190.00	266.00	17.72	61.20	47.03	29.50	25.85	1.35	15.28	...	5.38	991.90	1.02	353.953488	2
1	2024-01-01 01:00:00	191.00	283.00	14.15	62.95	45.03	34.02	27.30	1.27	19.40	...	5.32	991.92	1.00	354.720930	2
2	2024-01-01 02:00:00	183.75	278.50	8.10	67.80	42.65	25.95	26.90	1.20	16.50	...	5.28	991.92	0.92	349.156977	2
3	2024-01-01 03:00:00	182.50	272.75	3.92	66.45	38.55	20.00	24.25	1.23	15.63	...	5.15	991.90	0.95	348.197674	2
4	2024-01-01 04:00:00	193.75	279.75	7.00	65.68	40.60	23.25	26.08	1.35	11.75	...	5.08	991.90	0.90	356.831395	2

5 rows × 32 columns

```
df=dfi.drop(['Benzene (µg/m³)', 'Toluene (µg/m³)', 'Xylene (µg/m³)', 'O Xylene (µg/m³)', 'Eth-Benzene (µg/m³)', 'MP-Xylene (µg/m³)', 'TOT-RF (mg/m³)'])
df.head()
```



	Timestamp	PM2.5 (µg/m³)	PM10 (µg/m³)	NO (µg/m³)	NO2 (µg/m³)	NOx (ppb)	NH3 (µg/m³)	SO2 (µg/m³)	CO (mg/m³)	Ozone (µg/m³)	AT (°C)	RH (%)	WS (m/s)	WD (deg)	RF (mm)	SR (w/mt2)	
0	2024-01-01 00:00:00	190.00	266.00	17.72	61.20	47.03	29.50	25.85	1.35	15.28	10.50	75.22	0.32	177.00	NaN	5.38	96
1	2024-01-01 01:00:00	191.00	283.00	14.15	62.95	45.03	34.02	27.30	1.27	19.40	10.30	75.68	0.38	191.50	NaN	5.32	96
2	2024-01-01 02:00:00	183.75	278.50	8.10	67.80	42.65	25.95	26.90	1.20	16.50	10.22	75.75	0.45	191.25	NaN	5.28	96
3	2024-01-01	182.50	272.75	3.92	66.45	38.55	20.00	24.25	1.23	15.63	9.95	75.43	0.40	200.75	NaN	5.15	96

```
# Convert 'Datetime' column to datetime objects using the specified format "24-11-2017 17:00"  
# df['Time'] = pd.to_datetime(df['Timestamp'], format='%d-%m-%Y %H:%M')  
df['Time'] = pd.to_datetime(df['Timestamp'], format='%Y-%m-%d %H:%M:%S')
```

```
# Convert Datetime to numerical value (seconds since epoch)  
df['Datetime_num'] = df['Time'].astype(np.int64) // 10**9  
  
# Shift the AQI column 72 hours ahead (since the data is hourly, shift by 72 rows)  
df['target'] = df['AQI'].shift(-72)  
  
df=df.drop(['Timestamp'],axis=1)  
df=df.drop(['Time'],axis=1)  
  
df=df.drop(['Datetime_num'],axis=1)  
df=df.drop(['RF (mm)'],axis=1)  
df=df.dropna()  
df['PM2.5_to_PM10_ratio'] = df['PM2.5 (µg/m³)'] / df['PM10 (µg/m³)']  
df.head()
```



	PM2.5 (µg/m³)	PM10 (µg/m³)	NO (µg/m³)	NO2 (µg/m³)	NOx (ppb)	NH3 (µg/m³)	SO2 (µg/m³)	CO (mg/m³)	Ozone (µg/m³)	AT (°C)	RH (%)	WS (m/s)	WD (deg)	SR (w/mt2)	BP (mmHg)	VWS (m/s)
0	190.00	266.00	17.72	61.20	47.03	29.50	25.85	1.35	15.28	10.50	75.22	0.32	177.00	5.38	991.90	1.02
1	191.00	283.00	14.15	62.95	45.03	34.02	27.30	1.27	19.40	10.30	75.68	0.38	191.50	5.32	991.92	1.00
2	183.75	278.50	8.10	67.80	42.65	25.95	26.90	1.20	16.50	10.22	75.75	0.45	191.25	5.28	991.92	0.92
3	182.50	272.75	3.92	66.45	38.55	20.00	24.25	1.23	15.63	9.95	75.43	0.40	200.75	5.15	991.90	0.95
4	193.75	279.75	7.00	65.68	40.60	23.25	26.08	1.35	11.75	10.02	75.62	0.30	186.00	5.08	991.90	0.90

```
df.isnull().sum()
```



	0
PM2.5 (µg/m³)	0
PM10 (µg/m³)	0
NO (µg/m³)	0
NO2 (µg/m³)	0
NOx (ppb)	0
NH3 (µg/m³)	0
SO2 (µg/m³)	0
CO (mg/m³)	0
Ozone (µg/m³)	0
AT (°C)	0
RH (%)	0
WS (m/s)	0
WD (deg)	0
SR (W/mt2)	0
BP (mmHg)	0
VWS (m/s)	0
AQI	0
target	0
PM2.5_to_PM10_ratio	0




```
# === DATA CLEANING ===
# Maintain original preprocessing
df['target'] = df['AQI'].shift(-72)
df = df.dropna() # Remove rows with missing future values

# Create AQI category column
def categorize_aqi(aqi):
    if aqi <= 50: return 'good'
    elif aqi <= 100: return 'satisfactory'
    elif aqi <= 200: return 'moderate'
    elif aqi <= 300: return 'poor'
    elif aqi <= 400: return 'verypoor'
    else: return 'severe'

df['AQI_category_72h'] = df['target'].apply(categorize_aqi)
df = df.drop(['AQI', 'target'], axis=1)

# === FEATURE ENGINEERING ===
X = df.drop('AQI_category_72h', axis=1)
y = df['AQI_category_72h']

# Time-aware split (no shuffling)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False, random_state=42
)

# === MODEL TRAINING ===
clf = RandomForestClassifier(n_estimators=150,
                             max_depth=10,
                             class_weight='balanced',
                             random_state=42)

clf.fit(X_train, y_train)

# === FUTURE PREDICTIONS ===
# Get most recent available data
latest_data = X.tail(72) # Last 72 hours of sensor readings

# Predict next 72-hour AQI categories
future_predictions = clf.predict(latest_data)

# Create prediction timeline
```

```

# Create prediction timeline
prediction_times = pd.date_range(
    start=pd.to_datetime(df.index[-1]) + pd.DateOffset(hours=1),
    periods=72,
    freq='H'
)

# # Format results
# result = pd.DataFrame({
#     'Prediction_Time': prediction_times,
#     'Predicted_AQI_Category': future_predictions
# })

# print("72-Hour AQI Category Forecast:")
# print(result)

# Format results with proper datetime handling
result = pd.DataFrame({
    'Prediction_Time': prediction_times.strftime('%Y-%m-%d %H:%M:%S'),
    'Predicted_AQI_Category': future_predictions
})

# Configure pandas to show all rows
pd.set_option('display.max_rows', None)

print("72-Hour AQI Category Forecast:")
print(result.to_string(index=False))

```

```

↔ 72-Hour AQI Category Forecast:
  Prediction_Time Predicted_AQI_Category
1970-01-01 01:00:00      satisfactory
1970-01-01 02:00:00      moderate
1970-01-01 03:00:00      moderate
1970-01-01 04:00:00      satisfactory
1970-01-01 05:00:00      moderate
1970-01-01 06:00:00      moderate
1970-01-01 07:00:00      satisfactory
1970-01-01 08:00:00      moderate
1970-01-01 09:00:00      moderate
1970-01-01 10:00:00      moderate
1970-01-01 11:00:00      moderate
1970-01-01 12:00:00      satisfactory
1970-01-01 13:00:00      moderate
1970-01-01 14:00:00      moderate
1970-01-01 15:00:00      moderate
1970-01-01 16:00:00      moderate
1970-01-01 17:00:00      moderate
1970-01-01 18:00:00      moderate
1970-01-01 19:00:00      satisfactory
1970-01-01 20:00:00      satisfactory
1970-01-01 21:00:00      satisfactory
1970-01-01 22:00:00      satisfactory
1970-01-01 23:00:00      moderate
1970-01-02 00:00:00      moderate
1970-01-02 01:00:00      moderate
1970-01-02 02:00:00      moderate
1970-01-02 03:00:00      moderate
1970-01-02 04:00:00      moderate
1970-01-02 05:00:00      moderate
1970-01-02 06:00:00      moderate
1970-01-02 07:00:00      moderate
1970-01-02 08:00:00      moderate
1970-01-02 09:00:00      poor
1970-01-02 10:00:00      poor
1970-01-02 11:00:00      poor
1970-01-02 12:00:00      poor
1970-01-02 13:00:00      poor
1970-01-02 14:00:00      moderate
1970-01-02 15:00:00      poor
1970-01-02 16:00:00      moderate
1970-01-02 17:00:00      moderate
1970-01-02 18:00:00      moderate
1970-01-02 19:00:00      poor
1970-01-02 20:00:00      poor

```